Question Bank — Week 7

Queues (Lecture 07) — Answers

DSA27 · Data Structures and Algorithms

Fall 2026

Contents

Part A — Multiple choice **1**

Part B — Short answer and essay **4**

Part C — Trace the code **6**

Part D — Ring state **8**

Part E — Complexity analysis **9**

Part F — Find and fix the bug **9**

Part G — Write the code **10**

> ***Questions:*** *`week07-questions.md`. Commit to your own answer before reading one here.*

Part A — Multiple choice

Q	Ans	Q	Ans	Q	Ans	Q	Ans
M01	b	M11	a	M21	d	M31	c
M02	d	M12	c	M22	a	M32	b
M03	a	M13	d	M23	c	M33	d
M04	b	M14	c	M24	a	M34	a
M05	c	M15	a	M25	d	M35	c
M06	d	M16	b	M26	b	M36	b
M07	c	M17	d	M27	c	M37	d
M08	a	M18	a	M28	a	M38	c
M09	b	M19	c	M29	d	M39	a
M10	d	M20	b	M30	b	M40	b

W7-M01 — b. Last in, first out (a) is a stack — Week 6.

W7-M02 — d. The queue goes [1, 2, 3] → dequeue 1 → [2, 3] → enqueue 4 → [2, 3, 4] → dequeue 2 → [3, 4]; the next dequeue returns 3. (A stack would have returned 4 — the difference in one

question.)

W7-M03 — a. `pop(0)` moves the n − 1 elements behind the front. Amortised $O(1)$ (c) is the cost of append, the enqueue.

W7-M04 — b. `pop()` at the end moves nothing, but `insert_at(0, x)` shifts everything. With a plain array, whichever end is the front, one operation is $O(n)$.

W7-M05 — c. (6 + 3) % 8 = 9 % 8 = 1. Slot 9 (a) does not exist; the items are in slots 6, 7 and 0.

W7-M06 — d. (4 + 1) % 5 = 0: the head wraps. Slot 5 (a) is past the end of the `Array`.

W7-M07 — c. Empty: nothing between head and tail. Full: the tail has come all the way round to the head. Both look the same.

W7-M08 — a. With the size, empty is `size == 0` and full is `size == capacity`. The `Array` does know its length (c): `len(block)`.

W7-M09 — b. Slots 2 and 3 of the new block hold C and D, slots 4 and 5 are empty, and E and F sit in slots 0 and 1 — behind the front. The correct copy gives (a) with head = 0.

W7-M10 — d. Dequeue at the head (`pop_front`, $O(1)$), enqueue after the tail ($O(1)$).

W7-M11 — a. The new tail is the node before the old one, and a singly linked list can only find it by walking from the head.

W7-M12 — c. The first dequeue finds `outbox` empty and pours all three: `outbox` holds 3, 2, 1 (1 on top). It pops 1, leaving 2 and 3.

W7-M13 — d. One dequeue can pour n items, so the worst case (c) is $O(n)$, but each item is pushed, poured and popped once in its life.

W7-M14 — c. BFS visits nodes in the order they were discovered: FIFO. The other three are stack applications from Week 6.

W7-M15 — a. Dequeue number i shifts n − i elements: $(n-1)+(n-2)+\cdots+0 = n(n-1)/2$.

W7-M16 — b. Python's `%` takes the sign of the divisor, so the result is in 0..7: stepping back from slot 0 wraps to slot 7. In C or Java the answer would be −1 (a).

W7-M17 — d. A double-ended queue. (a) is a priority queue.

W7-M18 — a. Both are $O(1)$; the ring pays more per operation. Big-O hides constant factors, and measurement shows them.

W7-M19 — c. B, D, A and E leave in that order (W7-C2); C is last.

W7-M20 — b. Tail = 8: there is no slot 8, even though slots 0 to 4 are dead and free. That is why the queue must wrap.

W7-M21 — d. The occasional copy is $O(n)$, but doubling makes the copies total less than 2n over n enqueues — Lecture 04's argument.

W7-M22 — a. Same two operations, a different rule — and a different structure underneath, the heap.

W7-M23 — c. After four enqueues the ring is full (`1 2 3 4`, head 0). Two dequeues move head to 2, size 2. The tail is `(2 + 2) % 4 = 0`, so 5 goes into slot 0, freed by 1. Slot 4 (b) does not exist: that is the missing `%`. Head 0 (a) forgets that dequeue moves the head; size 4 (d) counts the dequeued items.

W7-M24 — a. The three dequeues leave D, E in slots 3 and 4, with head 3. F goes to `(3 + 2) % 5 = 0`, G to slot 1, and slot 2 is still empty. (b) is `list(q)`, the queue order, not the block. (d) forgets that `dequeue` clears the slot it read, so C is gone. (c) has seven slots in a five-slot block.

W7-M25 — d. `_grow` copies in queue order: C D E F go to slots 0–3 and head becomes 0. Then G is written at `(0 + 4) % 8 = 4`. (a) is what happens if head is kept, which is the naive grow of W7-M09. (b) would overwrite the front.

W7-M26 — b. The tail is still computed modulo 4: `(0 + 4) % 4 = 0`, so G overwrites C, the front, and `list(q)` reads `G D E F G`. No error is raised, which is what makes the bug dangerous: (c) is wrong. (a) is wrong because the arithmetic, not the block, decides where G goes.

W7-M27 — c. The four dequeues read slots 0–3, so they work, and head ends at 4. The enqueue writes E at `(4 + 0) % 4 = 0`, which also works. Then the dequeue reads `self._block[4]`, and the `Array` raises `IndexError`. The bug hides until the head has gone once round the ring: (b) is what the correct code does.

W7-M28 — a. After the first dequeue, head is `None` but tail still refers to the removed node 1. So `enqueue(2)` takes the `else` branch and links 2 after that dead node, and head stays `None`. The next dequeue finds head `None` and raises, although `len` says 1. The fix: when the queue becomes empty, reset `tail` to `None` too (W7-B4).

W7-M29 — d. Each item costs at most four operations: push to `inbox`, pop from `inbox`, push to `outbox`, pop from `outbox`. Items 1–4 are dequeued, so 4 × 4 = 16. Item 5 was poured but not dequeued, so 3 more: 19. Step by step: 3 pushes; a pour of 3 items (6) and 1 pop; 2 pushes; 2 pops; a pour of 2 (4) and 1 pop. (b) and (a) forget to count the pours as two operations per item; (c) is not a real count.

W7-M30 — b. One slot always stays empty, so that `head == tail` can only mean empty. Eight slots hold seven items. (a) would need a size field, which is what `dsa/queue.py` keeps.

W7-M31 — c. The size is `(tail - head) % 8 = (2 - 3) % 8 = 7`, and full is defined as `(tail + 1) % 8 == head`: `3 == 3`. With one slot left unused, 7 is the most an 8-slot ring can hold, so (d) is impossible. (a) reads the distance the wrong way round; (b) confuses `tail + 1 == head` (full) with `head == tail` (empty).

W7-M32 — b. The slot is outside the live part of the ring, so the queue is correct without clearing it (a, c and d are wrong: `is_empty` uses `_size`, `enqueue` uses `(head + size) % capacity`, and `__iter__` walks only `size` slots from head). But the block would keep a reference to the object, and Python's garbage collector could not free it.

W7-M33 — d. A fixed block that overwrites its oldest entry is exactly a ring buffer. Brackets (a) and undo (b) need stacks; sorting (c) needs neither.

W7-M34 — a. The queue gives breadth-first order: the children of `1` (10, 11) are dequeued before the children of `10` (100, 101), and those before the children of `11` (110, 111). That is the

binary numbers 1 to 6, in order. (b) follows one branch only (depth-first); (d) swaps 100 and 101; (c) enqueues `s + "1"` first.

W7-M35 — c. Queue 1 2 3 4 5 6: move 1, 2, remove 3. Queue 4 5 6 1 2: move 4, 5, remove 6. Queue 1 2 4 5: move 1, 2, remove 4. Queue 5 1 2: move 5, 1, remove 2. Queue 5 1: move 5, 1, remove 5. 1 is left. (a) removes 2 before 4: it loses track after the queue wraps.

W7-M36 — b. The stack hands back the most recently found node first, so the search goes as deep as it can before backing up. That is why Week 14 puts `bfs` and `dfs_iterative` side by side: the loops are the same, and only the structure differs. (c) cannot happen as long as visited nodes are not added again.

W7-M37 — d. `popleft()` removes the front in $O(1)$. `deque.pop()` (b) removes the **back**, and `deque.pop(0)` (a) is an error: `pop` takes no argument. On a `list`, `pop(0)` would work but costs $O(n)$. `deque` has no `dequeue` method (c).

W7-M38 — c. When a reading arrives, subtract the value that leaves the window and add the new one: $O(1)$ per update. Adding up k values each time would cost $O(k)$. The ring can be iterated (a), and `_size` tells full from empty (b).

W7-M39 — a. Slot of offset i is `(4 + i) % 6`, for i = 0 to 4: 4, 5, then round to 0, 1, 2. (b) forgets the `%`; (c) ignores head; (d) walks backwards.

W7-M40 — b. With head = 0, the i-th item of the queue really is in slot i, so copying slot by slot happens to give the right order. The naive copy goes wrong only once the data has wrapped: on the ring of W7-M25 it gives `C D None None G`. So (a) is wrong. A test must wrap the ring **before** it grows it.

Part B — Short answer and essay

W7-E1 *(4)*

- **ADT:** `enqueue(x)` at the back, `dequeue()` from the front (FIFO), `is_empty`, `len` — all $O(1)$; `dequeue` raises `IndexError` when empty.
- **Shifting:** on a plain array the front is a fixed position, index 0; removing it moves the n − 1 others: $O(n)$.
- **Swapping ends** does not help: the back at index 0 makes **enqueue** shift everything instead. A queue uses both ends, and an array is cheap at only one.
- **Ring buffer:** do not move data, move the indices. `head` marks the front; the back is `(head + size) % capacity`; both wrap round with `%`, so slots freed at the front are reused. Every operation $O(1)$.

W7-E2 *(4)*

- **Empty:** nothing in the ring, e.g. a new queue: head = tail = 0.
- **Full:** the tail has caught up with the head, e.g. capacity 4, head 2, four items: tail = (2 + 4) % 4 = 2.

- **Fixes:** (1) keep a count `size` — empty is `size == 0`, full is `size == capacity`; (2) leave one slot always unused — full is `(tail + 1) % capacity == head`, so `head == tail` means only empty; (3) a flag set when an enqueue makes head equal tail.
- `dsa/queue.py` keeps **`_size`** (and computes the tail from it).

W7-E3 *(4)*

- **Example:** capacity 4, head 2, block `[E, F, C, D]`, queue C D E F. A slot-by-slot copy into 8 slots keeps C, D in slots 2, 3 and E, F in slots 0, 1; with head 2 the queue now reads C, D, then empty slots — E and F are lost behind the front.
- **Correct copy:** in queue order: `new[i] = old[(head + i) % old_capacity]` for i in 0..size − 1, then head = 0 (and the capacity updated).
- **Amortised $O(1)$:** a copy happens only when the ring is full, and doubling means the copies after n enqueues total 1 + 2 + 4 + … < 2n: $O(n)$ over n operations, $O(1)$ each on average.
- It only goes wrong once the data has wrapped, which is why it survives tests that never dequeue before growing.

W7-E4 *(3)*

- `inbox` takes every enqueue; `dequeue` pops `outbox`, and when `outbox` is empty it first pours **all** of `inbox` into it — which reverses them, so the oldest is on top.
- **Amortised:** each item is pushed on `inbox` once, moved to `outbox` once and popped once — three $O(1)$ steps in its life, so n operations cost $O(n)$.
- **Pour only when empty:** pouring into a non-empty `outbox` would put newer items on top of older ones and break FIFO; pouring back and forth on every dequeue keeps FIFO but costs $O(n)$ per dequeue (W7-K3).

W7-E5 *(3)*

- **Ring buffer:** $O(1)$ (amortised if it grows); one contiguous block, no allocation per item, good locality. Choose it when the maximum size is known or bounded — hardware and OS buffers — or for speed in general.
- **Linked queue:** $O(1)$ worst case, never copies; but a node per item (more memory, scattered). Choose it when a worst-case guarantee matters more than memory.
- **`collections.deque`** combines them: a doubly linked list of fixed blocks — $O(1)$ at both ends, few allocations. It is what to use in real Python code.

W7-E6 *(4)*

- **No `%` on the tail:** once the back reaches the end of the block, the write goes to slot `capacity` and the `Array` raises `IndexError`, although there are free slots at the front. Shortest: `CircularQueue(2)`: enqueue, enqueue, dequeue, enqueue (tail = 1 + 1 = 2).
- **No `%` on the head:** dequeues work until head reaches `capacity`; the next dequeue reads slot `capacity` and raises. Shortest: `CircularQueue(1)`: enqueue, dequeue (head = 1), enqueue (written to `(1 + 0) % 1 = 0`), dequeue.
- **`_capacity` not updated in `_grow`:** the new block is bigger, but every `%` still wraps at the old size, so the next enqueue after the grow overwrites the front, silently. Shortest: fill a ring of capacity 1, enqueue twice more.
- **Why the tests miss them:** while the ring has not wrapped, head is 0 and `head + size` is below the capacity, so `%` changes nothing. Every test must take the ring **round** at least

once, and grow it after it has wrapped.

W7-E7 *(3)*

- **Refuse (bounded):** raise an error, or make the producer wait, or drop data when full. Memory is fixed. Used for audio and network buffers, and keyboard buffers, where memory must not grow.
- **Grow:** allocate a block twice as big and copy in queue order; enqueue stays amortised $O(1)$. Used for general-purpose queues such as `collections.deque`.
- **`level_order` and `bfs`** need a growing queue: they cannot know in advance how many nodes will be waiting at once (a whole level of a tree, a whole frontier of a graph), and refusing would make the search fail.

W7-E8 *(4)*

- **enqueue:** make a node. If the queue is empty, it becomes both `head` and `tail`; otherwise set `tail.next` to it and move `tail` to it. $O(1)$.
- **dequeue:** `IndexError` if empty; otherwise take `head.value`, move `head` to `head.next`; if `head` is now `None`, set `tail = None` too. $O(1)$.
- **Front at the head:** removing at the head is $O(1)$; removing the last node of a singly linked list needs the node before it, an $O(n)$ walk. So we add at the tail and remove at the head.
- **Stale tail:** `tail` still refers to the removed node. If `enqueue` tests `tail` to detect emptiness, the new node is linked after that dead node and `head` stays `None`: `len` says 1, but the next dequeue raises. (If it tests `head`, the bug hides, but `tail` still keeps a dead node alive.)

W7-E9 *(3)*

- A queue hands items back in the order they were **discovered**: everything at one level is dequeued before anything at the next. From "1": dequeue 1, enqueue 10, 11; dequeue 10, enqueue 100, 101; dequeue 11, enqueue 110, 111…
- So the order is 1, 10, 11, 100, 101, 110, 111, …: the binary numbers in increasing order, level by level (by length).
- With a **stack**, the last item found comes out first: 1, 11, 111, 1111, … (with `s + "0"` pushed before `s + "1"`). The search goes deep along one branch and never comes back to 10. That is depth-first order, as in Week 14.

Part C — Trace the code

W7-T1

```
[1, 2, True, [3, 4, 5, 6]]
```

After 1 is dequeued, head = 1; 4 goes to slot 3 and 5 wraps to slot 0; 2 is dequeued (head = 2); 6 goes to slot (2 + 3) % 4 = 1. The block is **`[5, 6, 3, 4]`**: slot 0 holds 5, slot 1 holds 6, slot 2 holds 3, slot 3 holds 4 — and the queue reads 3 4 5 6 from head = 2. Full: size 4 = capacity.

W7-T2

Operation	inbox	outbox	Returns
enqueue 1	1		
enqueue 2	1 2		
enqueue 3	1 2 3		
dequeue		3 2	1 *(poured 3 items, then popped 1)*
enqueue 4	4	3 2	
dequeue	4	3	2
dequeue	4		3
enqueue 5	4 5		
dequeue		5	4 *(poured 2 items)*
dequeue			5

Stacks are shown bottom to top. The values come out 1, 2, 3, 4, 5: FIFO. Only two pours happened, one per time `outbox` ran dry.

W7-T3

Leaves	Queue after, front to back
C	D E F A B
F	A B D E
D	E A B
B	E A
E	A
A	

Order: **C, F, D, B, E, A** — A is last. Each round moves two people from the front to the back, then removes the third.

W7-T4

Dequeued	Queue after its two enqueues
1	10 11
10	11 100 101
11	100 101 110 111
100	101 110 111 1000 1001
101	110 111 1000 1001 1010 1011
110	111 1000 1001 1010 1011 1100 1101

Returned: `["1", "10", "11", "100", "101", "110"]`. Every string of length k is dequeued before any of length k + 1: the queue works level by level, exactly as breadth-first search does.

W7-T5

```
1 3 5 6 4 2
```

The odd numbers are printed as they leave the queue, in their original order (FIFO). The even numbers wait on the stack and come off in reverse (LIFO). One loop, two structures, two orders.

Part D — Ring state

W7-S1.

```
operation    block                    head  size
enqueue 10   [10, _,  _,  _,  _ ]      0     1
enqueue 20   [10, 20, _,  _,  _ ]      0     2
enqueue 30   [10, 20, 30, _,  _ ]      0     3
enqueue 40   [10, 20, 30, 40, _ ]      0     4
dequeue      [_,  20, 30, 40, _ ]      1     3     returns 10
dequeue      [_,  _,  30, 40, _ ]      2     2     returns 20
enqueue 50   [_,  _,  30, 40, 50]      2     3
enqueue 60   [60, _,  30, 40, 50]      2     4     wrapped to slot 0
enqueue 70   [60, 70, 30, 40, 50]      2     5     full
dequeue      [60, 70, _,  40, 50]      3     4     returns 30
```

`list(q)` is **`[40, 50, 60, 70]`**; the queue is **not** full (size 4 of 5).

W7-S2.

```
enqueue 1, 2, 3   [1, 2, 3]   head 0, size 3     tails were 0, 1, 2: all fine
dequeue            [_, 2, 3]   head 1, size 2      returns 1
enqueue 4          tail = 1 + 2 = 3  ->  block[3]: IndexError (no slot 3)
```

Without `%`, the tail never wraps: slot 0, freed by the dequeue, is never reused. Every earlier enqueue happened while head was 0, where `head + size` is always less than the capacity, so it was correct. The bug only appears once the queue has both dequeued and filled up again — exactly the case this test was written for.

W7-S3.

```
operation        block            head  size   front to back
add_back 1       [1, _, _, _]      0     1     1
add_back 2       [1, 2, _, _]      0     2     1 2
add_front 0     [1, 2, _, 0]     3    3    0 1 2        head (0 - 1) % 4 = 3
add_front 9      [1, 2, 9, 0]      2     4     9 0 1 2        full
remove_back      [1, _, 9, 0]      2     3     9 0 1          returns 2
add_back 5      [1, 5, 9, 0]     2    4    9 0 1 5       slot (2 + 3) % 4 = 1
remove_front     [1, 5, _, 0]      3     3     0 1 5          returns 9
add_front 7      [1, 5, 7, 0]      2     4     7 0 1 5        full
add_back 8       [7, 0, 1, 5, 8, _, _, _]   0   5   7 0 1 5 8
```

Python's `%` gives a result from 0 to capacity − 1 even for −1, so stepping back from slot 0 lands on slot 3 (in C or Java, write `(head - 1 + capacity) % capacity`). `remove_back` needs no change to `head`: shrinking `size` moves the back. The last step grows first: the four items are copied **in deque order** into slots 0–3 of an 8-slot block (`head` becomes 0), then 8 goes to slot (0 + 4) % 8 = 4.

Part E — Complexity analysis

W7-K1 — $\Theta(n^2)$. The n enqueues are amortised $O(1)$ each: $\Theta(n)$. The dequeue that finds k items shifts k − 1 of them: $(n-1)+(n-2)+\cdots+0 = n(n-1)/2$. **With `CircularQueue(n)`:** every operation is $O(1)$ and no growth is needed, so $\Theta(n)$ in total.

W7-K2 — $\Theta(n^2)$. After the i-th enqueue the queue holds i items, and `sum(q)` iterates over all of them: $1 + 2 + \cdots + n = n(n+1)/2$. The queue operations are not the problem; the full walk inside the loop is. Keeping a running total instead makes it $\Theta(n)$.

W7-K3 — $\Theta(n^2)$ against $\Theta(n)$. When k items are in the queue, each dequeue moves all k to `outbox` and the k − 1 left back to `inbox`: about 2k moves, so the n dequeues cost about $2(n + (n-1) + \cdots + 1) \approx n^2$. The correct version pours once — all n items — and every later dequeue is a single pop: $\Theta(n)$ in total. (Measured with this course's `Stack`: 6,000 enqueues mixed with 2,000 dequeues took about 16 seconds with the pouring-back version; the correct version runs the W7-C1 test — 200,000 enqueues and 66,666 dequeues — in under a second.)

Part F — Find and fix the bug

W7-B1. `pop()` removes the **last** item — the newest — so this is a stack, not a queue: enqueue 1, 2, 3 and dequeue returns 3. `test_queue_is_fifo` fails. **Fix:** `return self._items.pop(0)` — $O(n)$, which is the point of `SlowQueue`.

W7-B2. `head` never wraps. `test_circular_queue_wraps_around` still passes because `__iter__` applies `% capacity` itself, but in `test_circular_queue_interleaved_operations` (capacity 4) the fifth dequeue reads `block[4]`: `IndexError`, although the queue holds items. **Fix:** `self._head = (self._head + 1) % self._capacity`.

W7-B3. It tests fullness by `tail == head`, which is also true when the queue is **empty**: the very first enqueue on a new queue raises "queue is full", and the queue can never hold anything. **Fix:** `if self._size == self._capacity:` — the size is what tells full from empty.

W7-B4. When the last node is removed, `head` becomes `None` but `tail` still points at the removed node. The next enqueue sees a non-empty `tail`, links the new node after the dead one, and leaves `head` at `None`: `enqueue(1)`, `dequeue()`, `enqueue(2)`, `dequeue()` raises "dequeue from empty queue". **Fix:** after moving `head`, `if self.head is None: self.tail = None`.

Part G — Write the code

W7-C1

```
class StackQueue:
    def __init__(self):
        self._inbox = Stack()              # enqueue pushes here
        self._outbox = Stack()             # dequeue pops here; oldest on top

    def enqueue(self, value):
        self._inbox.push(value)

    def dequeue(self):
       if self._outbox.is_empty():       # pour only when the outbox is empty
            while not self._inbox.is_empty():
                self._outbox.push(self._inbox.pop())
        return self._outbox.pop()          # IndexError when both are empty

    def __len__(self):
        return len(self._inbox) + len(self._outbox)
```

Why amortised $O(1)$: each value is pushed on `inbox` once, moved to `outbox` once, and popped once — three $O(1)$ steps over its lifetime, whatever the order of operations. A single dequeue can cost $O(n)$, but only after n cheap enqueues have paid for it.

W7-C2

```
def josephus(names, k):
    circle = CircularQueue(max(1, len(names)))
    for name in names:
        circle.enqueue(name)
    order = []
    while len(circle) > 0:
        for _ in range(k - 1):
            circle.enqueue(circle.dequeue())   # passed by: front to back
        order.append(circle.dequeue())          # the k-th leaves
    return order
```

The queue never holds more than it started with, so it never needs to grow. Cost: $O(nk)$ — each of the n removals first moves k − 1 people.

W7-C3

```
def moving_averages(values, k):
    if k < 1:
        raise ValueError("k must be at least 1")
    window = CircularQueue(k)
    total, result = 0, []
    for value in values:
```

```
        if len(window) == k:
            total -= window.dequeue()           # the oldest value leaves
        window.enqueue(value)
        total += value
        if len(window) == k:
            result.append(total / k)
    return result
```

Why $O(n)$: each value is enqueued once and dequeued at most once, and the running total changes by two $O(1)$ steps per value. Summing each window afresh would cost k additions per window: $O(nk)$ — 1.6 billion additions in the test with n = 100,000 and k = 20,000. The window never holds more than k values, so a `CircularQueue(k)` never needs to grow: a fixed-size ring buffer holding "the last k events" is exactly what ring buffers are for.

W7-C4

```
def reverse_first_k(values, k):
    if not 0 <= k <= len(values):
        raise ValueError(f"k must be between 0 and {len(values)}")
    queue = CircularQueue(max(1, len(values)))
    for value in values:
        queue.enqueue(value)
    stack = Stack()
    for _ in range(k):                          # the first k onto the stack...
        stack.push(queue.dequeue())
    result = []
    while not stack.is_empty():                 # ...come off reversed
        result.append(stack.pop())
    while len(queue) > 0:                       # the rest keep their order
        result.append(queue.dequeue())
    return result
```

The stack reverses what passes through it; the queue preserves it. $O(n)$.

W7-C5

```
def binary_numbers(n):
    result = []
    pending = CircularQueue(2 * n + 1)          # never needs to grow
    pending.enqueue("1")
    while len(result) < n:
        s = pending.dequeue()
        result.append(s)
        pending.enqueue(s + "0")
        pending.enqueue(s + "1")
    return result
```

Each dequeue adds two strings, so after n dequeues the queue holds n + 1: a capacity of 2n +

1 is more than enough. The strings come out in order because the queue finishes every length before starting the next — breadth-first, the idea behind Week 14's BFS. $O(n)$ queue operations (building each string also costs its length, $O(\log n)$).